INTERNATIONAL ISLAMIC UNIVERSITY MALAYSIA
DEPARTMENT OF MECHATRONICS ENGINEERING

MECHATRONICS SYSTEM INTEGRATION (MCTA 3203)

SEMESTER 1, 2026/2027

EXPERIMENT 2: SERIAL COMMUNICATION

SECTION 1

GROUP 10

NO	NAME	MATRIC NUMBER
1.	WIBOWOPUTRI ALYA SHAFIQA	2416728
2.	NURUL SYIFA BINTI RAZALI	2416110
3.	NURUL IZZAH KHALIESA BINTI MOHD NIZAM	2414774
4.	SHARIFAH NUR AZZARIENA EMILY BT SYED MOHD AZAM	2417458

SUBMISSION DATE: 18 OCTOBER 2026

ABSTRACT

This experiment aims to test real-time data collection and actuator control using a potentiometer, servo motor, LED and USB serial interface with the purpose to study serial communication between Arduino microcontroller and a Phyton application. The objectives were to use serial commands to drive the servo motor, send the potentiometer readings from the Arduino to Python and improve the system with real-time feedback and visualization.

The experimental procedure involved programming the Arduino IDE to read analog input from the potentiometer and send the values to Python through Serial USB communication at a baud rate of 9600 bps. Control commands were received, displayed and transmitted by Python using the pyserial library matplotlib to show the graphical visualisation of sensor data. The servo motor angle was adjusted based on user input and potentiometer readings and an LED provided visual feedback based on predetermined threshold conditions.

The system produced accurate servo positioning between 0° and 180°, stable potentiometer readings over the 0-1023 ADC range and dependable real-time communication with a response time suitable for interactive control. The classification accuracy was irrelevant because no classification algorithm was used. The main challenges encountered are delay in Python terminal display, LED control difficulties, real-time response performance, baud rate mismatches and hardware power limitations. Future improvements include error handling, use timestamp measurements, software debouncing, more efficient servo control algorithms, improve Python graph settings, review LED control logic, use more advanced visualisation interface and optimize serial communication to improve reliability and responsiveness.

TABLE OF CONTENT

TASK 1 : READING POTENTIOMETER VALUES VIA SERIAL COMMUNICATION..... 5
1.0 INTRODUCTION AND THEORY.....5
1.1 INTRODUCTION.....5
1.2 THEORETICAL BACKGROUND.....5
1.2.1 POTENTIOMETER OPERATING PRINCIPLE.....5
1.2.2 SERIAL COMMUNICATION.....5
1.2.3 FIRMWARE LOGIC.....5
1.2.4 COMPUTER-SIDE PYTHON SCRIPT.....6
1.3 EXPECTED TECHNICAL OUTCOMES.....6
2.0 MATERIAL AND EQUIPMENT.....6
3.0 EXPERIMENTAL SETUP AND HARDWARE ASSEMBLY.....6
4.0 METHODOLOGY AND PROGRAM LOGIC.....7
4.1 CIRCUIT ASSEMBLY AND HARDWARE CONFIGURATION.....7
4.2 PROGRAM LOGIC AND INTER-SYSTEM COMMUNICATION.....7
4.3 PROGRAM FLOW.....8
4.3.1 ARDUINO FIRMWARE.....8
4.3.2 PYTHON HOST SCRIPT.....8
4.4 ARDUINO C++ FIRMWARE.....8
4.5 PYTHON HOST SCRIPT.....9
5.0 DATA COLLECTION AND TECHNICAL ANALYSIS.....9
5.1 DATA COLLECTION.....9
5.2 TECHNICAL ANALYSIS.....10
6.0 RESULTS AND ENGINEERING DISCUSSION.....12
6.1 RESULTS.....12
6.2 ENGINEERING DISCUSSION.....12
7.0 CONCLUSION.....13
1.0 INTRODUCTION AND THEORY.....14
1.1 INTRODUCTION.....14
1.2 THEORETICAL BACKGROUND.....14
1.2.1 POTENTIOMETER OPERATING PRINCIPLE.....14
1.2.2 SERVO MOTOR OPERATION.....14
1.2.3 SERIAL COMMUNICATION BETWEEN ARDUINO AND PYTHON.....15
1.2.4 LED FEEDBACK.....15
1.2.5 REAL TIME GRAPHICAL VISUALIZATION.....15
2.0 MATERIAL AND EQUIPMENT.....15
3.0 EXPERIMENTAL SETUP AND HARDWARE ASSEMBLY.....16
4.0 METHODOLOGY AND PROGRAM LOGIC.....17

4.1 EXPERIMENTAL PROCEDURE..... 17
4.2 PROGRAM LOGIC AND INTER-SYSTEM COMMUNICATION..... 18
4.3 PROGRAM FLOW..... 19
4.4 ARDUINO C++ FIRMWARE..... 19
4.5 PYTHON HOST SCRIPT..... 20
5.0 DATA COLLECTION AND TECHNICAL ANALYSIS..... 23
5.1 DATA COLLECTION..... 23
5.2 MATHEMATICAL ANALYSIS..... 24
5.3 THEORETICAL DATA TABLE..... 25
6.0 RESULTS AND ENGINEERING DISCUSSION..... 27
6.1 EXPERIMENTAL RESULTS..... 27
6.2 ENGINEERING DISCUSSION AND LIMITATIONS..... 28
7.0 CONCLUSION..... 28
FINAL SYNTHESIS SECTION..... 29
1.0 RECOMMENDATIONS..... 29
2.0 REFERENCES..... 29
3.0 APPENDICES..... 31
4.0 ACKNOWLEDGMENT..... 33
5.0 STUDENT DECLARATION..... 33

TASK 1 : READING POTENTIOMETER VALUES VIA SERIAL COMMUNICATION

1.0 INTRODUCTION AND THEORY

1.1 INTRODUCTION

Serial communication is one of the most crucial methods used in embedded systems to exchange data between a microcontroller and a computer. In this experiment, an Arduino board reads a potentiometer's analog output and uses a USB serial connection to send the measured values to a Python software. Python receives and displays the transmitted data, enabling real-time monitoring of the potentiometer's position. For monitoring and control purposes, this communication shows how sensor data can be collected, processed and displayed on a computer.

The objective of this experiment is to establish serial communication between Arduino and Python, obtaining analog sensor data from a potentiometer and displaying the communicated values on a computer. The expected outcome of this experiment is a better understanding of the sensor interfacing, analog data collection, serial data transfer and the development of basic Python programs for the real-time monitoring are expected to be the results of this project.

1.2 THEORETICAL BACKGROUND

1.2.1 POTENTIOMETER OPERATING PRINCIPLE

A three-terminal variable resistor that divides voltage is called a potentiometer. A variable output voltage is provided by the center terminal (wiper) while the two outside terminals are connected to the power supply (5V) and ground (GND). The output voltage continually shifts between 0V and 5V as the potentiometer knob is turned. By using its integrated Analog-to-digital Converter (ADC), the Arduino reads this voltage through analog input pin A0 and transforms it into a digital value that the microcontroller can compute.

1.2.2 SERIAL COMMUNICATION

Serial communication is a method of transmitting data one bit at a time between two devices. In this experiment, a USB serial connection running at a baud rate of 9600 bps establishes communication between Arduino and a computer. The Python software receives the transmitted data using the pyserial library and the Arduino uses the Serial.println() function to send the potentiometer readings. The Arduino and Python programs need to use the same serial port and baud rate in order for communication to be successful.

1.2.3 FIRMWARE LOGIC

The Arduino firmware continuously executes inside the loop() function. It uses analogRead(A0) to read the potentiometer value during each cycle, transforms the analog signal into a digital value and then uses Serial.println() to send the reading through the serial port. A

short delay is included to prevent data from being transmitted too quickly. This ensures stable communication with the computer.

1.2.4 COMPUTER-SIDE PYTHON SCRIPT

The Python program uses the pyserial library to establish a serial connection with the Arduino through the selection COM port. It reads incoming serial data continuously, converts the received bytes into text and shows the potentiometer values in Python terminal. The received data can be used for monitoring, data logging and further processing. The experiment also points out that since only one program can use the serial port at a time, the Arduino Serial Plotter shouldn't be opened concurrently.

1.3 EXPECTED TECHNICAL OUTCOMES

This experiment aims to establish serial communication for real-time data exchange between an Arduino board and a Python application over a USB connection. The experiment also aims to interface a potentiometer with the Arduino to acquire analog sensor readings and transmit these readings continuously to a computer. Additionally, a Python program is created to accept and show the serial data. This allows the potentiometer output to be monitored in real time. Overall, the experiment demonstrates the integration of the sensors, microcontrollers and computer applications for data collection and embedded system monitoring.

2.0 MATERIAL AND EQUIPMENT

- Arduino Board
- Potentiometer
- LED
- 220 Ω resistor
- Jumper wires
- Breadboard

3.0 EXPERIMENTAL SETUP AND HARDWARE ASSEMBLY

1. Connect the Arduino board to the computer using a USB cable.
2. Connect the potentiometer to the Arduino:
    - Outer terminal 1 to 5V
    - Outer terminal 2 to GND
    - Centre terminal to analog pin A0
3. Connect the LED in series with a 220 Ω resistor to the Arduino circuit.
    - Connect the LED anode (longer leg) to digital pin 8 through a 220 Ω resistor.
    - Connect the LED cathode (shorter leg) to GND.
4. Connect the components using jumper wires and a breadboard to ensure stable connections.

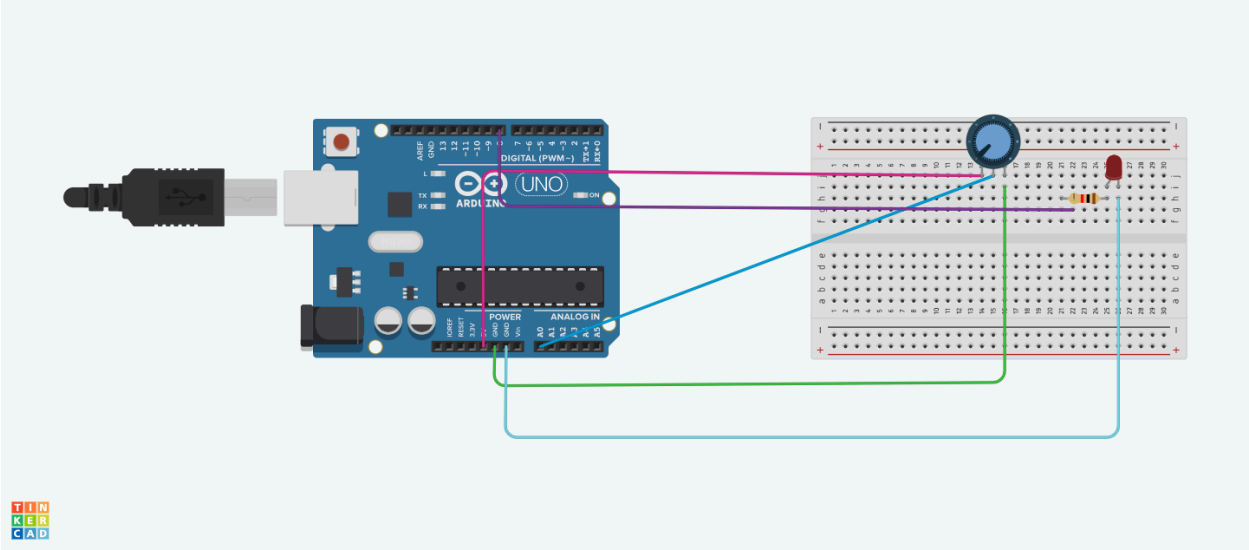


Figure 3.1 Circuit Design Schematics using Tinkercad

4.0 METHODOLOGY AND PROGRAM LOGIC

4.1 CIRCUIT ASSEMBLY AND HARDWARE CONFIGURATION

- The hardware components were assembled on a breadboard following Figure 3.1.
- Connect the Arduino Uno to the computer using a USB cable to provide power and establish serial communication.
- Upload the Arduino C++ firmware through the Arduino IDE.
- Open the Python host script and configure the correct COM port, COM5 with a baud rate of 9600 bps.
- Read the potentiometer value continuously using the analogRead() function.
- Transmit the potentiometer value to the computer through USB serial communication.
- Create the Python host script using the PySerial library.
- Open the correct COM port at 9600 bps in the Python script.
- Receive and decode the serial data sent by the Arduino.
- Display the potentiometer readings in the Python terminal.
- Rotate the potentiometer knob and observe the changing values displayed in real time to verify successful communication.

4.2 PROGRAM LOGIC AND INTER-SYSTEM COMMUNICATION

- The potentiometer provides an analog input voltage to the Arduino.
- The Arduino reads the analog value from A0 using the analogRead() function.
- The measured value (0–1023) is transmitted through USB serial communication using Serial.println().
- The communication baud rate is set to 9600 bps on both the Arduino and Python programs.

- The Python host script opens the corresponding COM port using the PySerial library.
- Python continuously waits for incoming serial data from the Arduino.
- The received data is decoded from bytes into text.
- The potentiometer value is displayed in the Python terminal for real-time monitoring.
- The communication process is one-way, where Arduino acts as the transmitter and Python acts as the receiver.

4.3 PROGRAM FLOW

4.3.1 ARDUINO FIRMWARE

1. Start the program.
2. Initialize serial communication (9600 bps).
3. Read the potentiometer value from analog pin A0.
4. Send the potentiometer value to the computer through the serial port.
5. Wait for 1 second.
6. Repeat Steps 3–5 continuously.

4.3.2 PYTHON HOST SCRIPT

1. Start the program.
2. Import the PySerial library.
3. Open the serial port which is COM5, 9600 bps.
4. Wait for incoming serial data.
5. Read one line of data from the Arduino.
6. Decode the received serial data.
7. Display the potentiometer value in the terminal.
8. Repeat Steps 4–7 continuously.
9. Close the serial port when the program is terminated.
10. End the program.

4.4 ARDUINO C++ FIRMWARE

```
const int potPin=A0;
const int ledPin=8;

void setup() {
Serial.begin(9600);
PinMode(LedPin)
```

```
}
void loop() {
int potValue = analogRead(A0);
Serial.println(potValue);
delay(1000);
}
```

4.5 PYTHON HOST SCRIPT

```
import serial
ser = serial.Serial('COM5', 9600)
try:
    while True:
        pot_value = ser.readline().decode().strip()
        print("Potentiometer Value:", pot_value)

except KeyboardInterrupt:
    ser.close()
    print("Serial Connection Closed")
```

5.0 DATA COLLECTION AND TECHNICAL ANALYSIS

5.1 DATA COLLECTION

Data collection was carried out by connecting a potentiometer to the Arduino Uno and transmitting its analog readings to a computer through USB serial communication. The potentiometer was connected to the 5 V supply, ground, and analog input pin A0. An LED was connected to digital pin D9 through a 220 Ω current-limiting resistor.

The Arduino read the potentiometer input using the `analogRead()` function and transmitted the readings to Python at a baud rate of 9600 bps. The Python program displayed the readings in the terminal and controlled the LED according to a threshold value of 512.

Measurements were collected at different potentiometer positions to observe the relationship between the input readings and the LED output state.

The following table illustrates the expected observations.

Potentiometer Reading	Input Voltage (V)	LED State

0	0.00	OFF
256	1.25	OFF
511	2.50	OFF
512	2.50	ON
768	3.75	ON
1023	5.00	ON

Table 5.1 The potentiometer readings, input voltage and the state of the LED

The graph illustrates the relationship between the potentiometer reading and the input voltage. As the potentiometer reading increases, the input voltage increases proportionally, ranging from approximately 0 V to 5 V. This linear relationship demonstrates the analog-to-digital conversion process performed by the Arduino Uno.

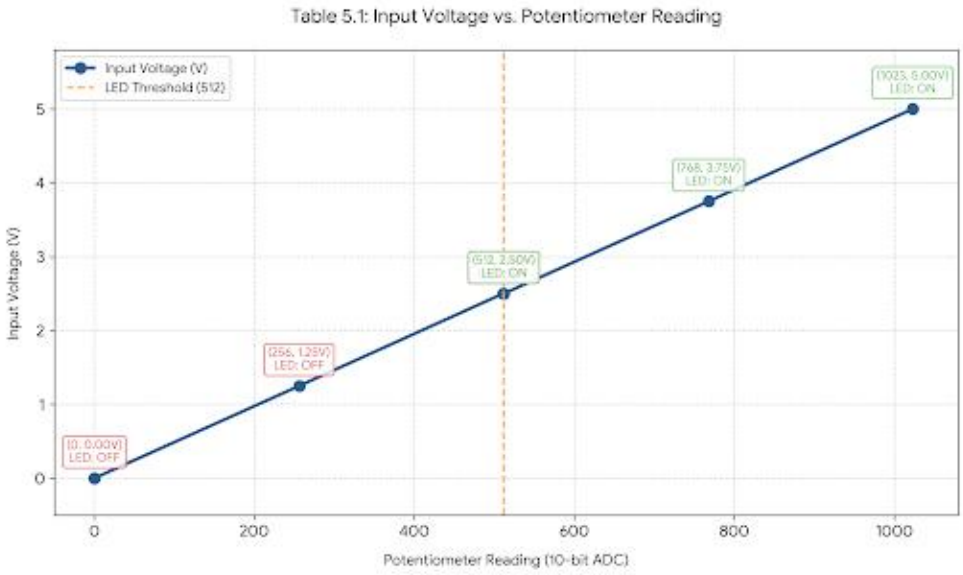


Figure 5.1 Input Voltage vs Potentiometer Reading Graph

5.2 TECHNICAL ANALYSIS

The experiment involves the measurement of analog input voltage using a potentiometer and the transmission of the measured data from an Arduino Uno to a Python application through USB serial communication. The potentiometer acts as a voltage divider, producing a variable output voltage depending on the position of its knob.

The Arduino Uno uses a 10-bit analog-to-digital converter (ADC) to convert the input voltage into a digital reading ranging from 0 to 1023. Assuming a reference voltage of 5 V, the input voltage can be estimated using the following equation:

$$Vi = (N \div 1023) \times Vref$$

For example, when the potentiometer reading is 700, the estimated input voltage is:

$$Vi = (700 \div 1023) \times 5 = 0.684V$$

$$= 3.42V$$

The relationship between the potentiometer reading and the input voltage is approximately linear. As the potentiometer knob is rotated, the output voltage changes, resulting in a corresponding change in the digital reading. This relationship is illustrated by the line graph of potentiometer reading against input voltage.

The measured digital readings are transmitted from the Arduino to Python through serial communication at a baud rate of 9600 bps. Python receives the readings, converts them into integer values, and compares them with a predefined threshold of 512. When the reading is greater than or equal to 512, Python sends an ON command to the Arduino, causing the LED connected to digital pin D9 through a 220 Ω resistor to illuminate. When the reading is below 512, an OFF command is sent, and the LED remains off.

The threshold value of 512 corresponds to approximately half of the ADC's full-scale range and an input voltage of approximately 2.50 V. Therefore, the LED control system demonstrates how analog input data can be processed through conditional programming to control a digital output automatically.

Overall, the experiment demonstrates the application of analog-to-digital conversion, serial data transmission, voltage analysis, and threshold-based control. The accuracy of the results may be influenced by variations in the reference voltage, electrical noise, potentiometer adjustment, and the stability of the serial communication connection.

6.0 RESULTS AND ENGINEERING DISCUSSION

6.1 RESULTS

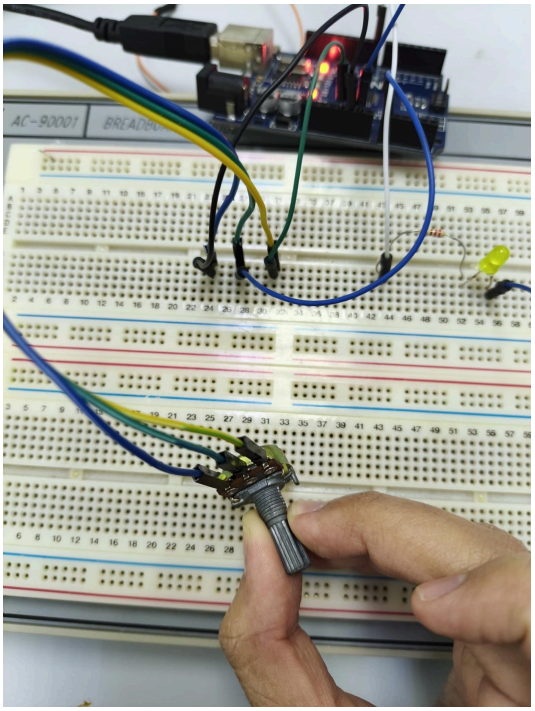


Figure 6.1 Succesful Operation of Communication between Arduino Uno and the Python Application

The experiment demonstrated successful communication between the Arduino Uno and the Python application through USB serial connection. As the potentiometer knob is rotated, the analog readings displayed in the Python terminal change according to the input voltage.

When the potentiometer reading is below 512, the Python program sends an OFF command to the Arduino and the LED remains off. When the reading reaches or exceeds 512, the program sends an ON command, causing the LED to illuminate.

The result shows that the potentiometer can be used as an input device and that Python can process the received data to control a physical output through the Arduino.

The experiment is considered successful when the potentiometer readings are displayed correctly in Python and the LED responds according to the specified threshold condition.

6.2 ENGINEERING DISCUSSION

The experiment illustrated the integration of hardware and software in an embedded control system. The potentiometer provides a variable analog input, the Arduino converts this input into a digital reading, and Python processes the data to determine the appropriate LED output.

The threshold-based control method is a simple example of decision-making in engineering systems. By comparing the input reading with a predefined threshold, the program can automatically switch an output device without requiring manual control of the LED.

The use of serial communication also demonstrates how sensor data can be transferred from a microcontroller to a computer for monitoring and processing. This approach can be extended to applications such as data logging, sensor monitoring, and automated control systems.

During the initial setup, a direct connection between the potentiometer and the Arduino could not be used because it prevented the Arduino from powering up correctly. This issue was addressed by checking the circuit configuration and verifying all wiring connections before testing. This highlights the importance of correct circuit design and proper wiring in ensuring the reliable operation of electronic systems.

Several factors may affect the experimental results. Small variations in analog readings can occur due to electrical noise, changes in the supply or reference voltage, and variations in the potentiometer position. Communication errors may also occur if the incorrect COM port is selected or if the baud rates do not match. Furthermore, the Arduino Serial Monitor or Serial Plotter should be closed when Python is accessing the same serial port to prevent communication conflicts.

One limitation of this implementation is that the LED changes state at a fixed threshold. If the input fluctuates around the threshold, the LED may switch rapidly between ON and OFF. This behaviour could be reduced by introducing hysteresis or suitable filtering in a more advanced control system.

7.0 CONCLUSION

In conclusion, this experiment demonstrated the integration of hardware and software in an embedded control system using an Arduino, a potentiometer, an LED, and Python. The potentiometer provided variable analog input, which was converted into digital readings by the Arduino and transmitted to Python through USB serial communication. The specific outcome of the experiment was that the potentiometer readings were displayed in the Python terminal, and the LED was programmed to turn ON when the reading reached or exceeded 512 and turn OFF when the reading was below 512. This demonstrated the successful implementation of threshold-based control using Python and Arduino.

The experiment also highlighted the importance of proper circuit configuration, accurate wiring connections, and compatible serial communication settings in ensuring reliable system operation. The initial power-up issue emphasized the need for systematic troubleshooting when working with electronic components.

Overall, this experiment provided practical experience in analog-to-digital conversion, serial communication, conditional programming, and hardware interfacing. These fundamental concepts can be applied to more advanced mechatronics applications, including sensor monitoring, data

logging, and automated control systems.

TASK 2 : CONTROLLING SERVO MOTOR VIA SERIAL COMMUNICATION

1.0 INTRODUCTION AND THEORY

1.1 INTRODUCTION

This experiment focuses on using serial communication between an Arduino Uno and Python. A potentiometer is used to control the position of a servo motor. The Arduino reads the potentiometer value and sends the reading to Python through a serial connection.

Python receives the data and displays it in the terminal and on a real time graph using Matplotlib. An LED is also included to provide visual feedback based on the potentiometer value. This experiment helps students understand how hardware and software can communicate and work together.

The main objective of this experiment is to control the angular position of a servo motor using a potentiometer while monitoring the sensor readings through Python. Additionally, the system is enhanced with an LED to provide visual feedback based on the potentiometer reading or servo position. A Matplotlib graph is also implemented in Python to visualize the potentiometer readings in real time. A keyboard command is included to allow the user to terminate the system operation when required.

The expected outcome is a functioning system that demonstrates the integration of sensor input, actuator control, LED feedback, serial communication and graphical data visualization.

1.2 THEORETICAL BACKGROUND

1.2.1 POTENTIOMETER OPERATING PRINCIPLE

A potentiometer is a three terminal variable resistor that produces a variable output voltage according to the position of its rotating knob. The two outer terminals are connected to the 5V supply and ground (GND) while the centre terminal is connected to the Arduino analog input pin A0.

The Arduino Uno uses a 10-bit analog-to-digital converter (ADC) to convert the input voltage into a digital reading ranging from 0 to 1023. A reading near 0 represents a voltage close to 0 V, while a reading near 1023 represents a voltage close to 5 V. These readings are used to determine the corresponding servo motor angle and LED output.

1.2.2 SERVO MOTOR OPERATION

A servo motor is an actuator that controls angular position according to a commanded angle. In this experiment, the servo motor signal wire is connected to digital pin 9 of the Arduino, while its power and ground wires are connected to the supply and GND connections.

The Arduino Servo library is used to control the motor through the write() function. The potentiometer reading is converted into an angle ranging from 0° to 180° using the map() function. When the potentiometer knob is rotated, the Arduino reads the new analog value and updates the servo motor position accordingly.

The relationship between the potentiometer reading and the servo angle is approximately linear. A low potentiometer reading produces a small angle while a high reading produces a larger angle.

1.2.3 SERIAL COMMUNICATION BETWEEN ARDUINO AND PYTHON

Serial communication allows the Arduino Uno to send data to Python through a USB connection. In this experiment, the Arduino uses Serial.begin(9600) to set the communication speed to 9600 baud. The potentiometer reading is sent using Serial.println(potValue).

Python uses the PySerial library to receive the data through COM5 at the same baud rate. The received value is printed in the terminal and used to update the graph. The Arduino sends only the potentiometer value while the servo angle is calculated inside the Arduino program.

Both the Arduino and Python programs must use the same baud rate and the correct COM port must be selected. The Arduino Serial Monitor and Serial Plotter should remain closed while Python is using the same serial connection.

1.2.4 LED FEEDBACK

An LED is incorporated into the circuit to provide visual feedback based on the potentiometer reading or servo position. The Arduino processes the analog reading and determines the required LED output.

The LED can operate using this approach. First, it can turn ON when the potentiometer reading exceeds a specified threshold and turn OFF when the reading falls below that threshold. For a 10-bit ADC, a threshold near 512 represents approximately half of the input range. When the potentiometer reaches 512, the LED will light up.

1.2.5 REAL TIME GRAPHICAL VISUALIZATION

Matplotlib is used to display the potentiometer readings in real time. Python stores each received reading and assigns it a reading number. The graph shows the reading number on the x-axis and the potentiometer value on the y-axis. The y-axis ranges from 0 to 1023, which is the reading range of the Arduino Uno's 10-bit analogue input.

A horizontal dashed line at 512 is included as a reference for the LED threshold. The graph is updated as new data is received, allowing changes in the potentiometer value to be observed.

The graph allows the user to observe how the potentiometer readings change when the knob is rotated. This provides a visual representation of the input signal and helps demonstrate the relationship between sensor input and system response.

2.0 MATERIAL AND EQUIPMENT

- Arduino Board + USB cable

- Servo Motor
- Jumper Wires
- Potentiometer
- 220Ω resistor
- LED
- Breadboard

3.0 EXPERIMENTAL SETUP AND HARDWARE ASSEMBLY

The hardware connections are arranged as follows:

1. Arduino connection: Connect the Arduino Uno to the computer using a USB cable.

2. Potentiometer connection:
    - Connect one outer terminal to the Arduino 5V pin.
    - Connect the other outer terminal to GND.
    - Connect the centre terminal to analog input pin A0.
3. Servo motor connection:
    - Connect the signal wire to digital pin 9.
    - Connect the VCC wire to the appropriate 5 V supply.
    - Connect the GND wire to Arduino GND.
    - Ensure that the servo and Arduino have a common ground.
4. LED connection:
    - Connect the LED in series with a 220 Ω resistor.
    - Connect the LED circuit to digital pin 8 and GND.

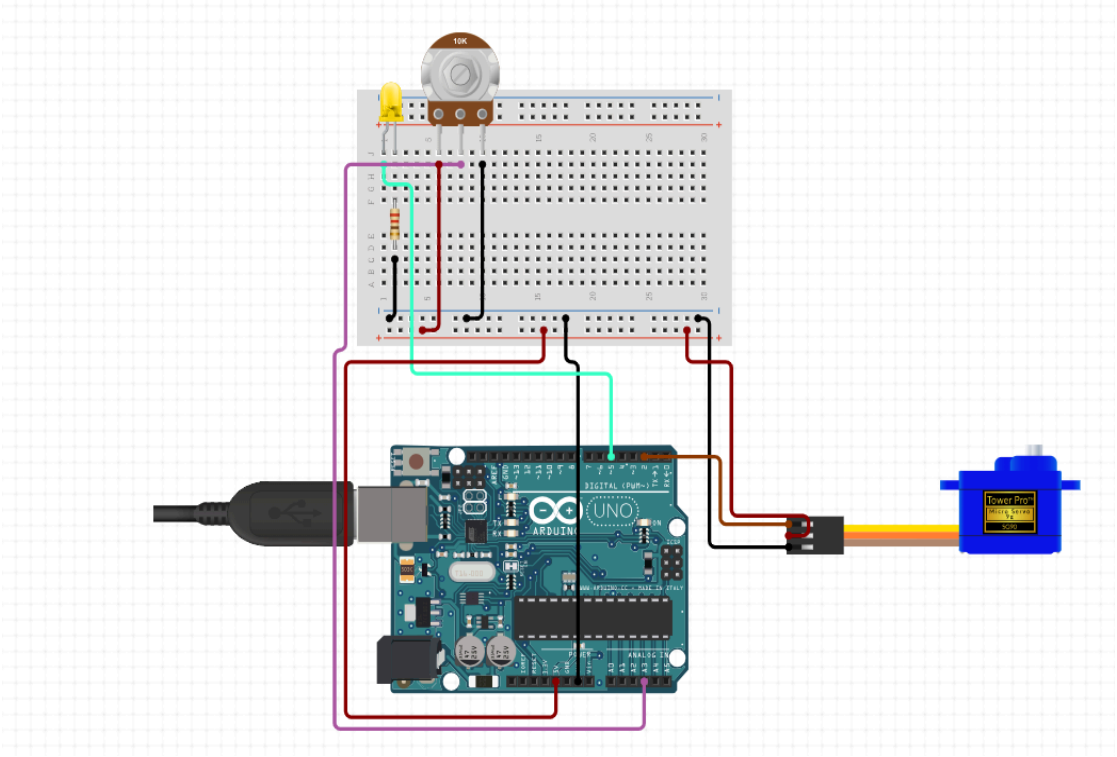


Figure 3.1 shows the circuit design schematic for Task 2

4.0 METHODOLOGY AND PROGRAM LOGIC

4.1 EXPERIMENTAL PROCEDURE

The experiment was conducted to integrate an Arduino microcontroller with a Python application for potentiometer monitoring, servo motor control, and LED feedback. The following procedures were performed:

1. The Arduino board was connected to the computer through a USB cable. The potentiometer was connected to analog pin A0, the servo motor signal wire to digital pin 9, and the LED to digital pin 8.
2. The Arduino program was uploaded using the Arduino IDE. The program initialized serial communication at 9600 baud, attached the servo motor, and configured the LED pin as an output.
3. The Python script was executed to establish serial communication through COM5. A two-second delay was included to allow the serial connection to initialize before data collection began.
4. The potentiometer was rotated to generate different analog readings. The readings were transmitted from the Arduino to Python and displayed in the Python terminal and real-time graph.

5. The servo motor response was observed as the potentiometer was rotated. The Arduino mapped the potentiometer readings to servo angles ranging from 0° to 180°.
6. The LED response was observed while adjusting the potentiometer. The Arduino mapped the potentiometer readings to LED output values ranging from 0 to 255 using pulse-width modulation (PWM).
7. The graph was monitored to observe changes in potentiometer readings. Although the Arduino IDE showed the expected response, a noticeable delay occurred in the Python terminal when the potentiometer was adjusted.
8. The experiment was concluded by stopping the Python plotting process and closing the serial connection.

4.2 PROGRAM LOGIC AND INTER-SYSTEM COMMUNICATION

The Arduino firmware and Python script performed complementary functions. The Arduino acquired the potentiometer readings and controlled the physical outputs, while Python received the transmitted readings and visualized them graphically.

At the beginning of the Arduino program, the Servo library was included, and the servo object was initialized. Serial communication was configured at 9600 baud, the servo was attached to pin 9, and pin 8 was configured as an output for the LED.

During each iteration of the main loop, the Arduino checked whether a serial command was available. If the character 'x' or 'X' was received, the program printed a termination message, moved the servo to 0°, switched off the LED, and entered an infinite loop. Otherwise, the Arduino reads the potentiometer value from analog pin A0.

The potentiometer reading was converted into a servo angle using the `map()` function, allowing the servo to move between 0° and 180°. The same reading was also mapped to a PWM output between 0 and 255 to control the LED brightness. The potentiometer value was then transmitted to the computer through serial communication, followed by a 50 ms delay before the next iteration.

On the computer, the Python script established a serial connection through COM5 at 9600 baud. It continuously read incoming data, converted valid readings into integers, and appended them to lists representing the reading sequence and potentiometer values. Matplotlib was used to update the graph interactively without clearing the entire figure during each update. A horizontal reference line at 511 was included to indicate the LED threshold reference.

The system successfully demonstrated potentiometer-based servo control, LED feedback, and graphical monitoring in Python. However, a delay was observed in the Python terminal during potentiometer adjustments, indicating that the monitoring response was not as immediate as the response observed in the Arduino IDE.

4.3 PROGRAM FLOW

The program logic can be summarized as follows:

1. Initialize the servo, LED output, and serial communication.
2. Check for a termination command from the computer.
3. Read the potentiometer value.
4. Convert the reading into a servo angle and update the servo position.
5. Convert the reading into a PWM value and adjust the LED brightness.
6. Transmit the potentiometer reading to Python.
7. Receive the reading in Python and update the graph.
8. Repeat the process while the programs are running.

4.4 ARDUINO C++ FIRMWARE

```C/C++
#include <Servo.h>

Servo myServo;

const int potPin = A0;     // Analog pin connected to potentiometer
const int servoPin = 9;    // Digital pin connected to servo motor
const int ledPin = 8;      // Digital pin connected to LED
indicator

void setup() {
  Serial.begin(9600);
  myServo.attach(servoPin);
  pinMode(ledPin, OUTPUT);
}

void loop() {
  // Verify presence of incoming serial data from host computer
  if (Serial.available() > 0) {
    char command = Serial.read();

    if (command == 'x' || command == 'X') {
      Serial.println("System operation terminated by user.");
```

```
      while (true) {
        // Halt system execution and return actuators to initial
positions
        myServo.write(0);       // Reset servo motor position to
zero degrees
        digitalWrite(ledPin, LOW);
        // Microcontroller remains in endless loop
      }
    }
  }

  // Acquire analog signal input from potentiometer (0-1023)
  int potValue = analogRead(potPin);

  // 1. Convert potentiometer readings to target servo position
angle (0-180 degrees)
  int servoAngle = map(potValue, 0, 1023, 0, 180);
  myServo.write(servoAngle);

  // 2. Adjust output signal duty cycle according to analog input
level
  // LED illumination intensity scales with measured input voltage
  int ledBrightness = map(potValue, 0, 1023, 0, 255);
  analogWrite(ledPin, ledBrightness);

  // 3. Transmit sensor data package to host application via USB
interface
  Serial.println(potValue);

  delay(50); // Provide short sampling interval for system
stability
}
```

4.5 PYTHON HOST SCRIPT

Python

```
import serial
import matplotlib.pyplot as plt
import time

# Establish serial communication link with Arduino microcontroller
ser = serial.Serial('COM5', 9600, timeout=1)
time.sleep(2)  # Provide delay for serial connection initialization

# Enable interactive plotting mode for real-time visualization
plt.ion()
fig, ax = plt.subplots()

x_vals = []
y_vals = []
count = 0

# Initialize plot trace for potentiometer signal data
line, = ax.plot([], [], label="Potentiometer Value")

# Include horizontal reference line denoting LED activation
threshold
ax.axhline(y=511, linestyle='--', label="LED Threshold")

ax.set_title("Real-Time Potentiometer Monitoring")
ax.set_xlabel("Time (readings)")
ax.set_ylabel("Potentiometer Value")
ax.set_ylim(0, 1023)
ax.legend()
ax.grid(True)

print("Turn the potentiometer to see the graph.")
print("Close the graph window or press Ctrl+C to stop.")

try:
    while True:
        # Acquire incoming sensor data stream via serial port
```

```
        data = ser.readline().decode(
            'utf-8', errors='ignore'
        ).strip()

        if not data:
            continue

        print(data)

        # Extract numeric potentiometer reading from data package
        if "Potentiometer:" in data:
            try:
                pot_value = int(
                    data.split("Potentiometer:")[1]
                    .split("|")[0].strip()
                )
            except (IndexError, ValueError):
                continue
        else:
            # Process raw numerical transmission from
microcontroller
            try:
                pot_value = int(data)
            except ValueError:
                continue

        # Record sampling instance index and analog value
        count += 1
        x_vals.append(count)
        y_vals.append(pot_value)

        # Update graphical output without re-rendering background
components
        line.set_data(x_vals, y_vals)
        ax.set_xlim(0, max(50, count))
```

```
        # Redraw figure frame dynamically
        fig.canvas.draw_idle()
        fig.canvas.flush_events()
        plt.pause(0.05)

except KeyboardInterrupt:
    print("Plotting stopped.")

finally:
    # Terminate serial interface session and release graphical
resources
    ser.close()
    plt.ioff()
    plt.close(fig)
    print("Serial connection closed.")
```

5.0 DATA COLLECTION AND TECHNICAL ANALYSIS

5.1 DATA COLLECTION

Potentiometer readings were collected using the Arduino's analog input pin A0, producing values from 0 to 1023. The readings were transmitted to Python through USB serial communication at 9600 baud. Python displayed the received values in the terminal and plotted them using Matplotlib, with the reading sequence on the horizontal axis and potentiometer value on the vertical axis.

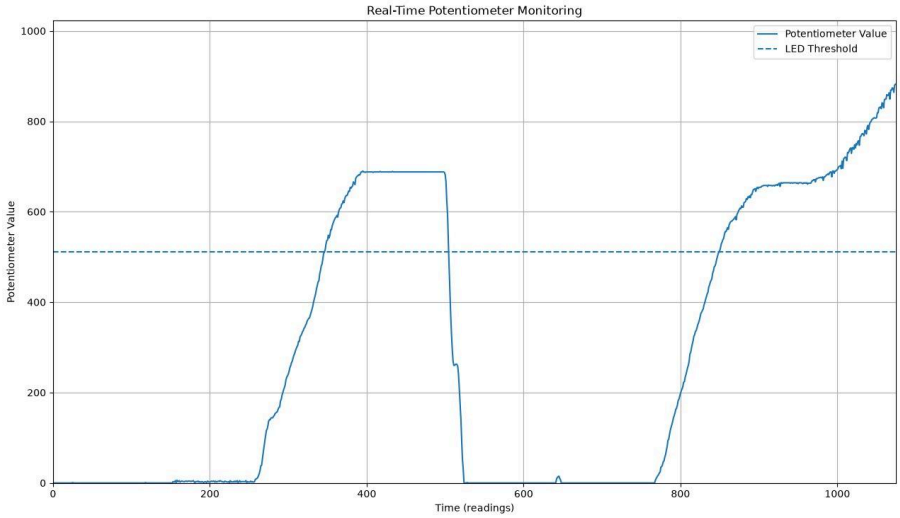


Figure 5.1 Real-Time Potentiometer Monitoring Using Python and Matplotlib

5.2 MATHEMATICAL ANALYSIS

The Arduino converted the potentiometer reading into a servo angle and LED PWM output using the `map()` function. The approximate relationships are:

1. Potentiometer reading to servo motor angle

$$\theta = \frac{180\,P}{1023}$$

Where:

- θ= servo motor angle in degrees (°)
- P= potentiometer reading (0–1023)

Example: If the potentiometer reading is 512,

$$\theta = \frac{180\,(512)}{1023}$$

$$\theta \approx 90.09°$$

Therefore, the servo motor angle is approximately 90°.

2. Potentiometer reading to LED PWM output

$$B = \frac{255\,P}{1023}$$

Where:

- B = LED PWM output value (0–255)
- P = potentiometer reading (0–1023)

Example: If the potentiometer reading is 512,

$$B = \frac{255\ (512)}{1023}$$

$$B \approx 127.63$$

Since the Arduino `map()` function returns an integer, the actual output is 127.

5.3 THEORETICAL DATA TABLE

Potentiometer reading	Servo angle (°)	LED PWM output
0	0	0
256	45	63
511	89	127
512	90	127
768	135	191
1023	180	255

Table 5.3 Theoretical outputs calculated from the Arduino program

The table shows that increasing the potentiometer reading increases the commanded servo angle and LED PWM output. These values are theoretical calculations rather than directly measured experimental data.

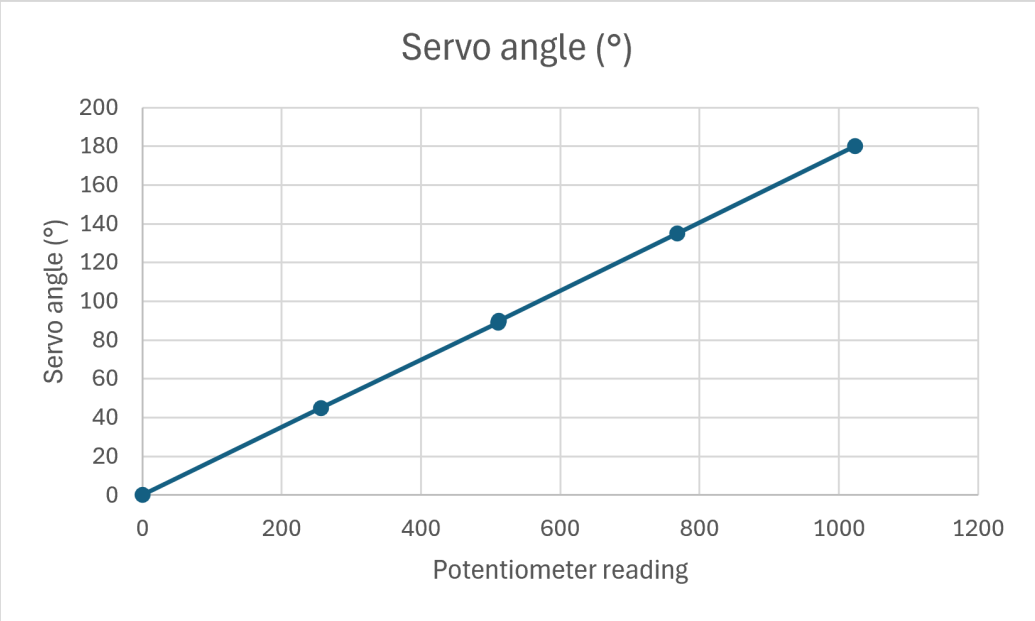


Figure 5.3.1 Potentiometer reading vs Servo angle(°)

The graph shows a positive linear relationship between the potentiometer reading and the servo motor angle. As the potentiometer reading increases from 0 to 1023, the commanded servo angle increases from 0° to 180°.

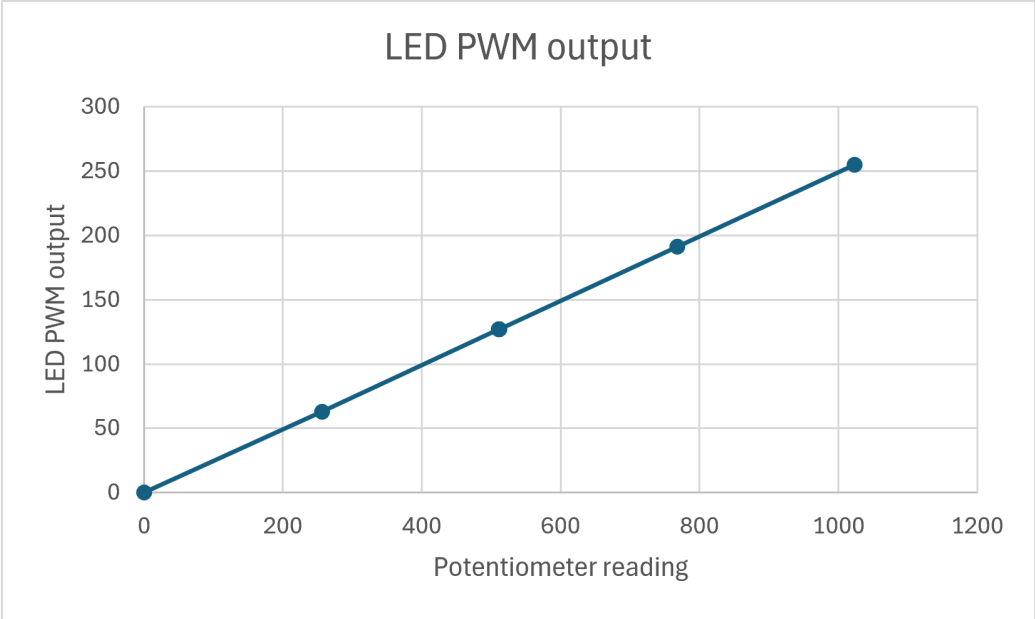


Figure 5.3.2 Potentiometer reading vs LED PWM output

The graph illustrates the relationship between the potentiometer reading and the LED PWM output. As the potentiometer reading increases, the PWM output increases from 0 to 255, allowing the LED brightness to be adjusted.

6.0 RESULTS AND ENGINEERING DISCUSSION

6.1 EXPERIMENTAL RESULTS

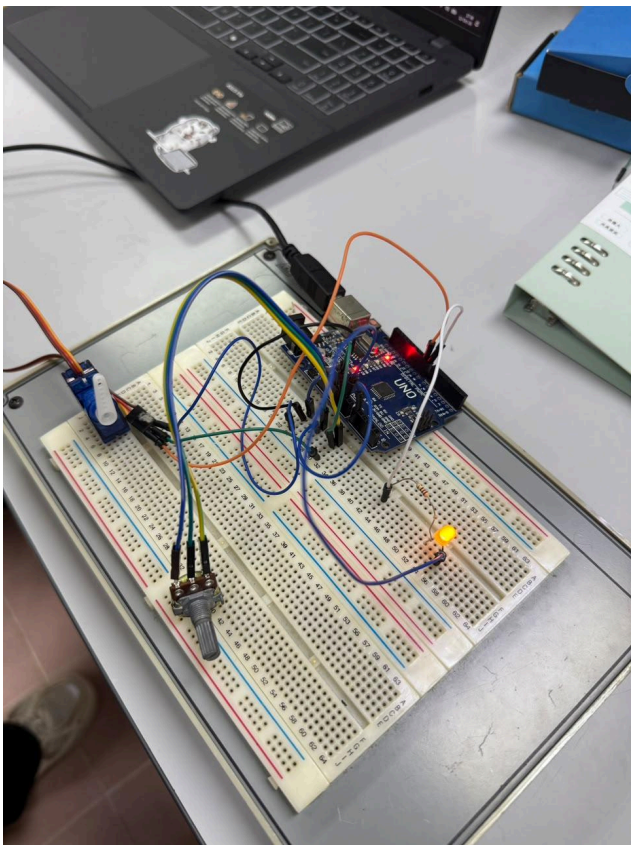


Figure 6.1 Successful Operation of the Potentiometer-Controlled Servo Motor and LED System

The experiment demonstrated successful integration of the Arduino hardware with the Python application. Rotating the potentiometer changed its analog readings, which were used by the Arduino to control the servo motor position and LED output. The servo motor responded to changes in the potentiometer position, while the LED provided visual feedback during operation.

The Python application successfully received the potentiometer readings and generated a graph using Matplotlib. As shown in Figure 5.1, the recorded values changed as the potentiometer was adjusted. The additional graphs in Figures 5.3.1 and 5.3.2 illustrated the relationships between the potentiometer reading, servo angle, and LED PWM output.

However, a noticeable delay occurred when updated readings were displayed in the Python terminal. Although the Arduino IDE responded as expected and the Python graph was generated successfully, the monitoring process did not provide an immediately updated display during potentiometer adjustments.

6.2 ENGINEERING DISCUSSION AND LIMITATIONS

The observed servo motor response demonstrated that the potentiometer could be used as an input for controlling the motor angle. Similarly, the LED provided an additional output that varied according to the potentiometer reading. These responses were consistent with the intended control logic implemented in the Arduino program.

The main limitation was the delay in the Python terminal. One possible contributing factor was the serial reading timeout of one second in the Python script. Other factors, such as serial data buffering and graph refresh processing, could also have affected the displayed response. However, the exact cause was not confirmed during the experiment.

To improve the system, the serial communication and graph update process could be optimized, and the delay settings could be adjusted and tested. The system could also be evaluated using recorded timestamps to measure the time between potentiometer changes and the corresponding updates in Python.

Another consideration is the LED control method. The Arduino program used pulse-width modulation (PWM) to vary LED brightness according to the potentiometer reading. Therefore, the intended behavior was continuous brightness adjustment rather than a strict ON/OFF response at a threshold of 512. The LED control logic should be checked against the intended experimental requirement to ensure consistent operation.

Overall, the experiment demonstrated the feasibility of integrating potentiometer input, servo motor control, LED feedback, and Python-based graphical monitoring. Despite the delay in displaying updated readings, the main hardware control functions and data visualization were successfully demonstrated.

7.0 CONCLUSION

In conclusion, the experiment successfully demonstrated the integration of an Arduino Uno, potentiometer, servo motor, LED, and Python through serial communication. The potentiometer provided analog readings that were mapped to control the servo motor's angular position and the LED output. The Python program successfully received the readings and displayed them in a real-time graph, demonstrating the connection between hardware control and software-based data visualization. However, a noticeable delay was observed in the Python terminal when the potentiometer was adjusted, indicating that the serial monitoring process could be improved. Overall, the experiment achieved its main objective of implementing a potentiometer-controlled system with real-time data monitoring and provided practical experience in microcontroller programming, serial communication, and hardware-software integration.

FINAL SYNTHESIS SECTION

1.0 RECOMMENDATIONS

To improve the reliability and performance of the system, several improvements are recommended. Firstly, the Serial Monitor in the Arduino IDE must be closed before running the Python program in PyCharm. Both applications cannot normally access the same Arduino serial port (COM5) simultaneously. If the Serial Monitor remains open, PyCharm may fail to establish a serial connection or display the potentiometer readings. Therefore, users should close the Serial Monitor before running the Python script and ensure that the correct COM port and baud rate of 9600 are configured in the Python code.

Secondly, the serial communication and graph update settings in Python should be optimized to reduce delays when displaying real time potentiometer readings. The graph refresh interval and serial datareading process should be adjusted to provide smoother monitoring without overloading the program.

Furthermore, all hardware connections should be checked before starting the experiment to ensure that the potentiometer, servo motor and LED are connected to the correct Arduino pins. The servo motor should also receive a suitable power supply, with a common ground maintained between the Arduino and the external supply if one is used.

In addition, the LED control logic should be checked to ensure that it responds correctly to the potentiometer reading. If the LED is designed to turn ON or OFF at a specific threshold, a clear threshold value should be defined in the Arduino code.

Finally, timestamps can be added to the Python program to record when potentiometer readings are received. This would help identify communication delays and evaluate the responsiveness of the system during real time monitoring.

2.0 REFERENCES

[1] Arduino. (n.d.). *analogRead()*. Arduino Language Reference. https://docs.arduino.cc/language-reference/en/functions/analog-io/analogRead/
[2] Arduino. (n.d.). *analogWrite()*. Arduino Language Reference. https://docs.arduino.cc/language-reference/en/functions/analog-io/analogWrite/
[3] Arduino. (n.d.). *Servo library*. GitHub. https://github.com/arduino-libraries/Servo/blob/master/docs/api.md
[4] PySerial. (n.d.). *API reference*. https://www.pyserial.org/docs/api-reference
[5] Matplotlib Development Team. (n.d.). *Interactive figures and asynchronous programming*. Matplotlib Documentation. https://matplotlib.org/stable/users/explain/figure/interactive_guide.html
[6] Arduino. (n.d.). *Arduino documentation*. https://docs.arduino.cc/
[7] International Islamic University Malaysia, Department of Mechatronics Engineering. (2026). *Mechatronics System Integration (MCTA 3203): Week 3—Serial Communication (Version 3.0)* [Laboratory manual].

3.0 APPENDICES

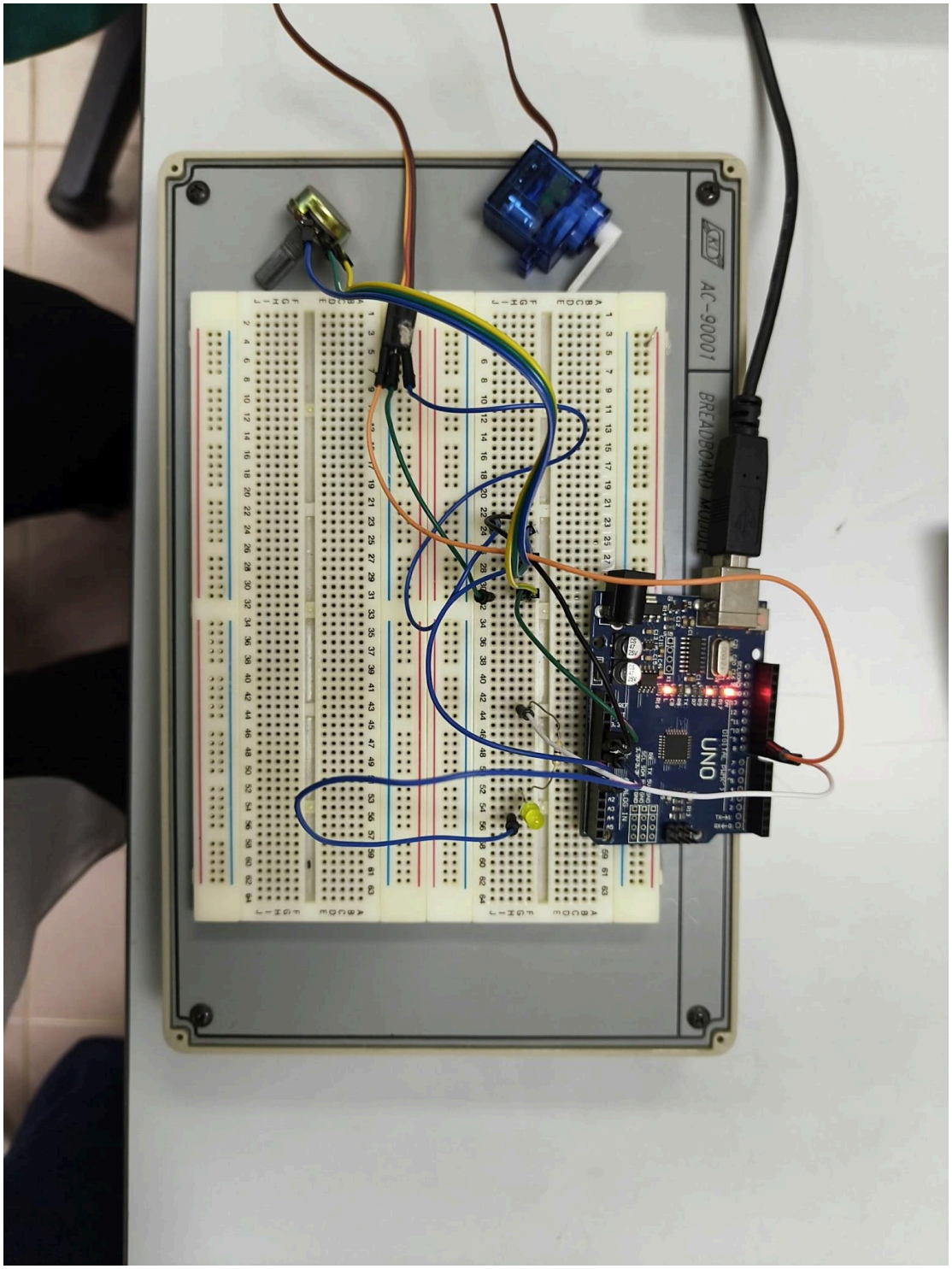


Figure 3.1 Hardware assembly of the Potentiometer-Controlled Servo Motor and LED System

4.0 ACKNOWLEDGMENT

First and foremost, we would like to express my sincere gratitude to Allah SWT for granting us the opportunity, strength and guidance to learn, complete this laboratory work and gain valuable knowledge throughout this experiment. Other than that, we would like to deeply thank Dr. Ahmad Imran Bin Ibrahim for sharing his valuable knowledge and providing continuous guidance throughout the completion of this laboratory work. His assistance and dedication to teaching have helped us gain a better understanding of the concepts covered in this laboratory. Last but not least, we would also like to extend our heartfelt thanks to our fellow friends for their cooperation and support throughout the completion of this laboratory assignment.

5.0 STUDENT DECLARATION

We hereby declare that the work presented in this technical report is entirely our own, except where explicitly acknowledged. We certify that in developing and completing this mechatronics project, we have strictly adhered to the standards of academic integrity and have not engaged in any form of plagiarism or unethical conduct. All sources of information, reference materials, and technical support utilized in this work have been appropriately cited and acknowledged.

Certificate of Originality and Authenticity

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgment, and that the original work contained herein has not been undertaken or done by unspecified sources or persons. We hereby certify that this report has not been done by only one individual, and all of us have contributed to the report. The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have read and understand the content of the total report, and no further improvement on the reports is needed from any of the individual contributors to the report. We therefore agreed unanimously that this report shall be submitted for marking, and this final printed report has been verified by us.

SIGNATURE	STUDENT DETAILS	CHECKLIST
	Name: WIBOWOPUTRI ALYA SHAFIQA Matric No: 2416728	Read [/] Understand [/] Agree [/]

	Name: NURUL SYIFA BINTI RAZALI Matric No: 2416110	Read [/] Understand [/] Agree [/]
	Name: NURUL IZZAH KHALIESA BINTI MOHD NIZAM Matric No: 2414774	Read [/] Understand [/] Agree [/]
	Name: SHARIFAH NUR AZZARIENA EMILY BT SYED MOHD AZAM Matric No: 2417458	Read [/] Understand [/] Agree [/]